计算机系统 I

期末复习讲义 & 配套真题巩固

知识点速览 + 章节配套练习（无答案，供自测）

使用说明：

- 每章先看 **“核心知识速览”** 快速回忆，再做 **“Exercises”** 检验。**练习全部为英文**（与期末考试一致），知识点中考试会出现的概念已用英文标注。
- 练习取自本课程真题卷、回忆卷及计组/数逻同知识点真题，已剔除超纲题。
- 标签：[**Exam**] 为 24–25 学年本课程期末卷原题；[**Recall**] 为历年回忆题；[**Extra**] 为配套同步练习；[**Class**] 取自老师习题讲解。
- 配套另有一页 A4 `cheat_sheet.pdf`，可直接誊抄进考场小抄。

涵盖 Lec01–Lec08：信息表示 · 布尔代数 · 组合逻辑 · 运算单元 · 时序逻辑 · ISA · RISC-V · 单/多周期 CPU

目录

1　信息表示 (Information Representation)

核心知识速览 (Key Concepts)

数制与转换 (Number Systems & Radix Conversion)

- **R 进制 → 十进制 (radix-r to decimal)**：按权展开 (weighted sum) $\sum A_i r^i$。
- **十进制 → R 进制**：整数部分“除基取余、逆序 (divide by radix, take remainders)”；小数部分“乘基取整、顺序 (multiply by radix, take integer parts)”。
- **二进制 ↔ 八/十六进制**：以小数点为界，每 3 位（octal）/ 4 位（hex）一组，不足补 0。

整数编码 (Integer Encodings): 原 / 反 / 补 / 移码

设 n 位带符号整数，最高位为符号位 (sign bit，0 正 1 负)。

- **原码 (sign-magnitude)**：符号位 + 绝对值，有 $+0/-0$。
- **反码 (one's complement, 1's)**：正数不变；负数除符号位外逐位取反 (bitwise NOT)。$= (r^n - 1) - N$。
- **补码 (two's complement, 2's)**：负数 = 反码 $+1$；$= r^n - N$。唯一的 0；范围 (range) $-2^{n-1} \le X \le 2^{n-1} - 1$（负数比正数多一个）。
- **快捷求补**：从最低位扫描，保留末尾的 0 与第一个 1，其后各位全部取反。恒等式 $\sim x + 1 = -x$。
- **移码 (biased / excess)**：真值 + 偏置常数 (bias)，等于补码符号位取反，常用于浮点阶码 (exponent)。
- **符号扩展 (sign extension)**：无符号补 0，有符号补符号位；**截断 (truncation)** 相当于对 2^k 取模。

IEEE 754 浮点数 (Floating Point)

$V = (-1)^s \times M \times 2^E$，存储为 `[s | exp | frac]`（sign / exponent / fraction）。

- **单精度 (single)** 32 位 = 1+8+23，Bias = 127；**双精度 (double)** 64 位 = 1+11+52，Bias = 1023。
- **规格化 (normalized)**（exp 不全 0 也不全 1）：$E = \exp - \text{Bias}$，$M = 1.\text{frac}$（隐含 1，implied leading 1）。
- **非规格化 (denormalized)**（exp 全 0）：$E = 1 - \text{Bias}$，$M = 0.\text{frac}$；表示 0 及接近 0 的数。
- **特殊值 (special values)**（exp 全 1）：frac 全 0 为 $\pm\infty$ (infinity)；frac 非 0 为 NaN。
- 默认舍入 (rounding)：**向偶数舍入 (round to nearest even)**（恰好一半时使最低有效位为 0）。

- 浮点加法**不满足结合律 (not associative)** $(x+y)+z \neq x+(y+z)$；相等判定危险，应用 ϵ。

非数值数据与存储 (Non-numeric Data & Storage)

- **BCD**：8421 有权码 (weighted code)；十进制每位用 4 位二进制表示（如 $34 \to 0011\,0100$）。余 3 码 (excess-3)、格雷码 (Gray code) 为无权码。
- **ASCII**：7 位；大小写字母相差第 6 位。Unicode 16 位。
- **字节序 (endianness)**：大端 (big-endian) MSB 放低地址；小端 (little-endian) LSB 放低地址（x86、RISC-V 主流）。
- **单位**：容量按 2^{10}（KiB/MiB/GiB）；频率/速率按 10^3。

Exercises

Exercises (Past Exams & Practice)

1. **[Quiz]** Represent decimal 75 as (a) a 7-bit binary number and (b) an 8-bit 8421-BCD code. Then place an even-parity bit at the MSB to form a 9-bit code.
2. **[Extra]** Given the bit pattern `0xAD100002`, interpret the value it represents (show your derivation) as each of: (1) a two's complement integer; (2) an unsigned integer; (3) a biased / excess representation; (4) a single-precision floating-point number.
3. **[Exam]** The BCD representation of decimal 34 is? A. `00110100` B. `0011 0100` C. `100010` D. `0010 0010`
4. **[Exam]** What is the 8-bit two's complement representation of $(-35)_{10}$?
5. **[Exam]** Write the IEEE 754 single-precision binary representation of -17.5625.
6. **[Extra]** Write the IEEE 754 single-precision binary representation of -23.3125_{10}.
7. **[Extra]** For the following nine 32-bit values, sort the labels in **ascending order** (sort only; you need not write out the exact values):

Label	Bit pattern	Interpretation
A	0xF0000000	sign-magnitude
B	0xF0000000	two's complement
C	0xF0000000	biased
D	0xF0000000	IEEE 754 float
E	0xF0000000	one's complement
F	0xF7000000	sign-magnitude
G	0xF7000000	two's complement
H	0xF7000000	biased
I	0xF7000000	IEEE 754 float

8. **[Extra]** Number conversion: $(17.5)_8 = (__)_{10} = (__)_{16}$.

9. **[Extra]** Write the **sign-magnitude / one's complement / two's complement** of the 8-bit machine number of $(-7.7)_{10}$ (1 sign bit + integer + 3 fraction bits).

10. **[Extra]** Let `x7` = `0x10000000`, and the data at that address is `0x1122334455667788`. Execute

```
lb x6, 0(x7)
sd x6, 8(x7)
```

and give the value stored at `0x10000008` on a **big-endian** machine and on a **little-endian** machine.

11. **[Class]** Convert each of the following numbers from its given base into the other three bases in the table (decimal / binary / octal / hexadecimal): (1) 369.3125_{10}; (2) 10111101.101_2; (3) $(326.5)_8$; (4) $(D6.A)_{16}$.

12. **[Class]** Find the radix r in each equation: (a) $(BEE)_r = (2699)_{10}$; (b) $(365)_r = (194)_{10}$. (Hint: expand by place value to form an equation, solve the resulting quadratic, and take the sensible positive integer root.)

13. **[Class]** Write the IEEE 754 representation of 63.25_{10} in (a) single precision (32-bit) and (b) double precision (64-bit). Show the sign, exponent, and fraction fields.

14. **[Class]** Given that `0x0C000000` is the bit pattern of an IEEE 754 **single-precision** float, find the real number it represents (show the derivation of sign, exponent, and mantissa).

15. **[Class]** A custom 16-bit floating-point format: 1 sign bit + 5 exponent bits (bias = 15) + 10 fraction bits. Write the binary representation of -1.5625×10^{-1} (i.e. -0.15625) in this format.

2　布尔代数与数字逻辑基础 (Boolean Algebra)

核心知识速览 (Key Concepts)

逻辑门与运算 (Logic Gates & Operations)

- 基本：AND $X{\cdot}Y$、OR $X+Y$、NOT $\overline{X}$。扩展：NAND $\overline{XY}$、NOR $\overline{X+Y}$。
- XOR $X\oplus Y=\overline{X}Y+X\overline{Y}$（不同为 1）；XNOR $\overline{X\oplus Y}=XY+\overline{X}\,\overline{Y}$（相同为 1）。
- n 输入真值表 (truth table) 有 2^n 行；真值表是函数**唯**一表示，表达式 (expression) 不唯一。

关键定理 (Key Theorems)

- 吸收律 (absorption) $X+XY=X$，　$X(X+Y)=X$；　合并 (combining) $XY+X\overline{Y}=X$。
- **一致性 (consensus)**：$XY+\overline{X}Z+YZ=XY+\overline{X}Z$（删冗余项 YZ）。
- **德摩根 (De Morgan)**：$\overline{XY}=\overline{X}+\overline{Y}$，　$\overline{X+Y}=\overline{X}\,\overline{Y}$。
- **对偶 (dual)**：交换 $+\leftrightarrow\cdot$、$0\leftrightarrow1$；对偶 $\neq$ 函数补 (complement，补还要对每个变量取反)。
- 优先级 (precedence)：括号 > NOT > AND > OR。

标准形式与化简 (Standard Forms & Minimization)

- **最小项 (minterm)** m_j：变量为 0 取反；**最大项 (maxterm)** M_j：变量为 1 取反。$m_j=\overline{M_j}$。
- 规范 SOP (sum of products) $=\sum m$(为 1 的行)；规范 POS (product of sums) $=\prod M$(为 0 的行)。两者唯一。
- **成本 (cost)**：文字成本 (literal cost) L；门输入成本 (gate-input cost) $G=L+$（非单文字项数）；$GN=G+$ 反相器数。
- **卡诺图 (Karnaugh map, K-map)**：圈大小为 2 的幂；每个 1 至少被圈一次；圈尽量大；可跨边界；无关项 (don't-care) 仅在有利时圈。
- **主蕴含项 (prime implicant, PI)**：不可再扩大的圈；**必要主蕴含项 (essential PI, EPI)**：覆盖了别的 PI 没覆盖的最小项。

Exercises

Exercises (Past Exams & Practice)

1. **[Quiz]** For the original expression

$$F=A(B+\overline{C})+\overline{A}(\overline{B}+C)+BCD+\overline{B}\,\overline{C}D,$$

calculate the gate-input cost, simplify F, and calculate the gate-input cost of the simplified expression. State clearly whether inverter inputs are included in your cost model.

2. **[Quiz]** For

$$F(A,B,C,D) = \sum m(1,3,7,11,15) + \sum d(0,2,5),$$

(a) draw the K-map; (b) list every prime implicant and every essential prime implicant; (c) obtain minimum SOP and POS expressions separately.

3. **[Extra]** Simplify the following using algebraic manipulation, and compute the gate-input cost before and after simplification:

$$F = D\overline{C} + C + ABC + AB\overline{C} + B\overline{C} + CD$$

4. **[Exam]** Simplify $F = A\overline{C} + ABC + AC\overline{D} + CD$. The result equals:
A. $A + CD$　B. $A\overline{C} + AC + CD$　C. $A + AC\overline{D} + CD$　D. $A\overline{D} + CD$

5. **[Exam]** Gate-cost model: each NOT counts as 1, each n-input gate counts as n. Find the total gate cost of the direct implementation of $F = AB\overline{D} + B + C(B + \overline{A}D)$.
A. 10　B. 12　C. 14　D. 16

6. **[Extra]** Find the gate-input cost G of $F = A + B\overline{C}(A\overline{B} + \overline{A}B)$.
A. 9　B. 10　C. 11　D. 12

7. **[Extra]** How many 1-cells does $AB\overline{C} + A\overline{D}$ occupy in a 4-variable Karnaugh map?
A. 13　B. 12　C. 6　D. 5

8. **[Extra]** Find the simplified complement $\overline{F}$ of:

$$F = \overline{AC + \overline{A}BC + \overline{B}\overline{C}} + AB\overline{C}$$

9. **[Extra]** Simplify using Boolean algebra (state the law used at each step):

$$F = (\overline{A}\,\overline{B} + \overline{A}B + A\overline{B})(\overline{A}C + \overline{B}C + AB)$$

10. **[Exam]** A 4-variable truth table is given below (X = don't-care).

A	B	C	D	Y	A	B	C	D	Y
0	0	0	0	0	1	0	0	0	X
0	0	0	1	1	1	0	0	1	0
0	0	1	0	X	1	0	1	0	0
0	0	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	0	1
0	1	0	1	X	1	1	0	1	1
0	1	1	0	0	1	1	1	0	X
0	1	1	1	0	1	1	1	1	1

(a) Write $Y = \sum m(\cdots) + \sum d(\cdots)$; (b) draw the K-map, find all PIs and EPIs, and write the minimal SOP; (c) use the K-map to find the minimal POS.

11. **[Extra]** For $F(A,B,C,D) = \sum m(0,1,2,3,6,8) + \sum d(10,11,12,13,14,15)$: (a) draw the K-map and write the product-of-maxterms form $\prod M(\cdots)$; (b) circle the essential prime implicants and list the corresponding product terms; (c) give the minimal SOP and POS.

12. **[Extra]** Use a K-map to simplify $F = \sum m(0,1,5,7,8,11,13) + \sum d(3,9,12,15)$ into the minimal **NAND-NAND** form (mark the largest groups and their product terms).

13. **[Extra]** How many discrete values can a binary logic variable take? A. 1 B. 2 C. 3 D. 4. How many output states does a three-state (tri-state) buffer have? When several tri-state buffer outputs are wired together, at most how many may be enabled at once?

14. **[Class]** The truth tables of functions E, F are given below:

X	Y	Z	E	F
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	1	0
1	1	1	0	1

(a) list the minterms and maxterms of E and F; (b) list the minterms of $\overline{E}, \overline{F}$; (c) list the minterms of $E + F$ and $E \cdot F$; (d) write E, F in algebraic sum-of-minterms form; (e) simplify E, F to expressions with the fewest literals.

15. **[Class]** Prove the following Boolean identities algebraically (state the theorem used at each step): (a) $\overline{A}C + A\overline{B} + B\overline{C} = \overline{A}C + A\overline{B}$ (consensus); (b) $(A+B)(\overline{A}+C)(B+C) = (A+B)(\overline{A}+C)$.

16. **[Class]** For the following functions with don't-cares, find all PIs and EPIs, and write the minimal SOP using the selection rules:
(a) $F(A,B,C) = \sum m(3,5,6) + \sum d(0,7)$;
(b) $F(W,X,Y,Z) = \sum m(0,2,4,5,8,14,15) + \sum d(7,10,13)$;
(c) $F(A,B,C,D) = \sum m(4,6,7,8,12,15) + \sum d(2,3,5,10,11,14)$.

3　组合逻辑设计 (Combinational Logic)

核心知识速览 (Key Concepts)

组合逻辑与设计流程 (Combinational Logic & Design Flow)

- 组合逻辑**无记忆 (memoryless)**：输出只依赖当前输入；无反馈环 (no feedback loop)、一个节点只一个驱动源。
- 设计流程：Specification → Formulation（真值表/方程）→ Optimization → Technology Mapping → Verification。

常用功能块 (Functional Blocks)

- **译码器 (decoder)** $n \to 2^n$：每个输出是一个最小项。n-to-2^n 译码器 + OR 门可实现任意函数（把为 1 行的输出 OR 起来）。带使能 (enable) 可作解复用器 (demultiplexer)。
- **编码器 (encoder)** $2^n \to n$；普通编码器多输入为 1 时输出歧义，需**优先编码器 (priority encoder)**（含 valid 有效位 V）。
- **多路选择器 (multiplexer, MUX)** 2^n-to-1：n 条选择线 (select lines) 选 1 路。用 2^n-to-1 MUX 实现 n 变量函数：输入接选择线，数据端按真值表置 0/1。
- **半加器 (half adder)** $S = X \oplus Y,\ C = XY$；**全加器 (full adder)** $S = X \oplus Y \oplus Z,\ C = XY + XZ + YZ$。
- 七段译码器 (seven-segment decoder)：共阳低有效，共阴高有效。

时序分析与 Verilog (Timing & Verilog)

- 传播延迟 (propagation delay) T_{pd}（最长路径，关键路径 critical path）；污染延迟 (contamination delay) T_{cd}（最短路径）。
- 竞争冒险 / 毛刺 (hazard / glitch)：同一输入经不同延迟路径到达输出产生短暂错误。
- Verilog：`module/endmodule`、`assign`（数据流 dataflow）、`always`（行为 behavioral）；`wire` 接线、`reg` 过程变量；可综合 (synthesizable) vs 仅仿真（testbench 延时/显示）。

Exercises

Exercises (Past Exams & Practice)

1. **[Exam]** Express the following using the Verilog conditional operator (`?:`); choose the equivalent form:

$$O = \overline{S_0}\,\overline{S_1} I_3 + \overline{S_0} S_1 I_2 + S_0 \overline{S_1} I_1 + S_0 S_1 I_0$$

A.
```
O = S0 ? (S1 ? I0 : I1) : (S1 ? I2 : I3);
```
B.
```
O = S1 ? (S0 ? I0 : I2) : (S0 ? I1 : I3);
```

C. `O = S0 & S1 ? I0 : I3;`

D. `O = S0 ? I0 : I1;`

2. **[Exam/Extra]** Using one 4-to-1 MUX (A, B on the select lines) plus a few external gates, implement

$$F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$$

(Hint: for each case $AB = 00, 01, 10, 11$, find F as a function of C, D.)

3. **[Exam]** Using only **two 3-to-8 decoders with enable**, **one inverter**, and **OR gates with at most 4 inputs**, implement $F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$. Draw the complete logic diagram.

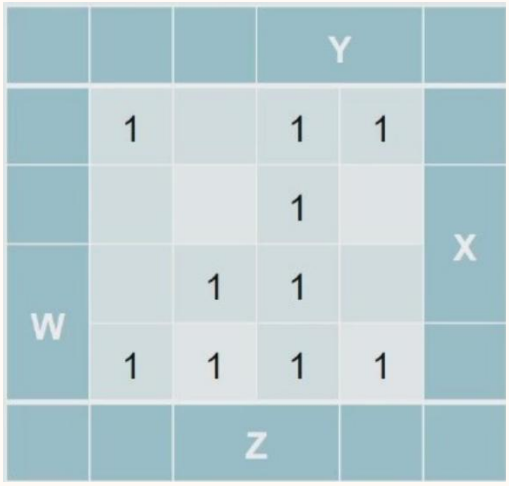
4. **[Extra]** (a) Design a 4-to-16 decoder using two 3-to-8 decoders and sixteen 2-input AND gates; (b) design a 4-to-16 decoder with enable using five 2-to-4 decoders with enable.

5. **[Extra]** Design a combinational circuit that compares two 4-bit unsigned numbers A, B and outputs X: $X = 1$ when $A < B$, otherwise $X = 0$.

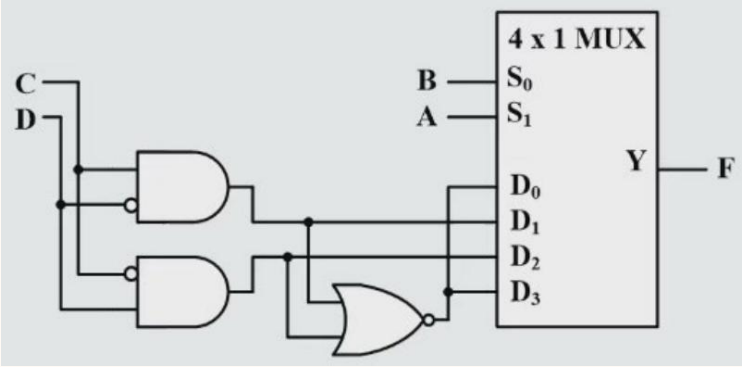
6. **[Extra]** 1-bit unsigned comparator: inputs A, B, outputs $X(A{>}B), Y(A{<}B), Z(A{=}B)$. (1) Build the truth table; (2) implement it using **one 2-to-4 decoder** plus gates; (3) using this 1-bit comparator as a module, design a 2-bit unsigned comparator from **two** such modules plus gates ($A[1{:}0], B[1{:}0]$, outputs $A{>}B, A{<}B, A{=}B$).

7. **[Extra]** Implement a 1-bit full adder $S = \sum m(1,2,4,7)$, $C = \sum m(3,5,6,7)$ using a 3-to-8 decoder plus OR gates.

8. **[Recall]** In the Karnaugh map shown below, how many prime implicants does the function have? A. 4　B. 5　C. 6　D. 7



9. **[Extra]** Analyze the logic circuit below (A, B on select lines S_1, S_0; C, D feed the data inputs $D_0 \sim D_3$ through external gates; output F): build the truth table of F, write the logic expression of F, and state what function this circuit can at most realize.



10. **[NumLogic]** Design a three-input majority voter with one 3-to-8 decoder and necessary gates. For inputs A, B, C, the output $F = 1$ exactly when at least two inputs are 1. Give the truth table, canonical Boolean function, and complete logic diagram.

4　算术运算单元 (Arithmetic Unit)

核心知识速览 (Key Concepts)

加法器与减法 (Adders & Subtraction)

- 串行进位加法器 (ripple-carry adder, RCA) $t = N\,t_{FA}$，简单但慢；超前进位 (carry-lookahead, CLA) 用 propagate $P_i = A_i \oplus B_i$、generate $G_i = A_i B_i$ 提前算进位：$C_{i+1} = G_i + P_i C_i$，约 $\log_2 N$ 级的前缀加法器 (prefix adder) 更快。
- 补码统一加减 (add/subtract)：S=1 时把 B 取反、C_0=1，得 $A + \overline{B} + 1 = A - B$。

溢出、进位与标志位 (Overflow, Carry & Flags)

- **无符号 (unsigned) 看进位 carry，带符号 (signed) 看溢出 overflow**（不可混用！）。
- 带符号溢出 (signed overflow)：正 + 正 = 负或负 + 负 = 正；判定 $OF = C_n \oplus C_{n-1}$（不能只看 C_n）。
- 标志位 (flags)：ZF（zero）、SF（sign）、CF（carry/borrow）、OF（signed overflow）。
- 无符号 m 位 × n 位，积 (product) 最多 $m + n$ 位。

乘法 / 除法 / 浮点 (Multiply / Divide / FP)

- 逐次乘法 (sequential multiply) = 移位 (shift) + 加法 (add)；**Booth 算法 (Booth's algorithm)** 把连续 1 压缩：遇 1（左邻 0）减，0（左邻 1）加，中间只移位；阵列乘法器 (array multiplier) 以面积换速度。
- 移位即乘除 2^N：$A \ll N = A \cdot 2^N$，算术右移 (arithmetic shift right) 保留符号位。
- 除法 (division)：恢复除法 (restoring) 试减失败要加回；非恢复除法 (non-restoring) 用下轮加法修正。
- 浮点加法 (FP add)：**对阶 (align exponents) → 加尾数 → 规格化 (normalize) → 检溢出 → 舍入 (round)**；浮点乘法 (FP multiply)：符号异或、阶码相加（减一次 bias）、尾数相乘、规格化、舍入。

Exercises

Exercises (Past Exams & Practice)

1. **[Quiz]** Using one 4-bit full adder and necessary gates, design a controllable 4-bit increment/decrement unit. For control $S = 0$ the circuit increments the 4-bit input by 1; for $S = 1$ it decrements the input by 1. Draw the circuit, label every adder input and carry, and state how overflow/carry should be interpreted.
2. **[Exam]** A combinational multiplier multiplies two 5-bit unsigned numbers. At most how

many bits does the product need?
A. 5 B. 8 C. 9 D. 10

3. **[Exam]** In two's complement arithmetic, when does overflow occur?
A. adding a positive and a negative number B. carry-out $c_n = 1$ C. the two operands have different sign bits D. $c_n \oplus c_{n-1} = 1$

4. **[Extra]** Use Booth's algorithm (radix-2 or radix-4) to compute $13 \times (-21)$; show the step-by-step process.

5. **[Extra]** A CPU takes 10 ns per multiply and 1 ns per subtract. How long does computing $d = a \times b - a \times c$ take? Can you rewrite the expression to make it faster?

6. **[Extra]** Use two's complement subtraction to compute $01010100_2 - 01000011_2$ and $01000011_2 - 01010100_2$; state the carry out of the MSB, whether correction is needed, and the final signed result.

7. **[Extra]** Given floats $s1 > s0 > 0$, `t3=0xFF000000`, `t4=0x00800000`. **Without using `fadd.s`**, implement $s2 = s1 + s0$ using only RV32I/RV64I integer instructions (i.e. perform exponent alignment, mantissa addition, and normalization by hand).

5　时序逻辑 (Sequential Logic)

核心知识速览 (Key Concepts)

存储元件 (Storage Elements)

- 时序电路 (sequential circuit) = 组合逻辑 + 存储元件；下一状态 (next state) = f(input, current state)。
- **NOR 型 SR 锁存器 (SR latch)** 高有效，禁止输入 (forbidden input) S=R=1；**NAND 型**低有效，禁止输入 S=R=0。
- D 锁存器 (D latch) 电平敏感 (level-sensitive, 透明 transparent)；**边沿触发 D 触发器 (edge-triggered D flip-flop)** $Q^+ = D$ 是同步设计标准。
- JK 触发器：11 翻转 (toggle)；T 触发器：1 翻转，$T = Q \oplus Q^+$。
- 主从触发器 (master-slave) 有 ones-catching 问题；建立时间 (setup time) t_s、保持时间 (hold time) t_h。

状态机分析与设计 (FSM Analysis & Design)

- **Moore 机**：输出只依赖现态（状态内标注，状态多更稳定，“Moore is More”）；**Mealy 机**：输出依赖现态 + 输入（边上标输入/输出，状态少）。
- 设计流程：spec → 状态图/表 (state diagram/table) → 状态编码 (state encoding) → 触发器输入方程 (flip-flop input equations) → 输出方程 → 优化 → 映射 → 验证。
- 状态编码位数 $n = \lceil \log_2 m \rceil$；相邻编码可简化卡诺图。
- **时序约束 (timing constraint)**：$t_p \geq t_{pd,FF} + t_{pd,COMB} + t_s$，决定最高频率 (max clock frequency)。

寄存器与计数器 (Registers & Counters)

- 移位寄存器 (shift register)：串入串出 (SISO) / 并行加载 (parallel load)（每级加 MUX 选保持/移位/加载）。
- 同步计数器 (synchronous counter) 共用时钟；波纹计数器 (ripple counter) 仅最低位接时钟，高位被前级触发，延迟 $\approx n\,t_{PHL}$。
- 模 N 计数器 (mod-N counter)：推荐**检测 $N-1$ 后同步清零/加载 (synchronous clear/load)**；**检测 N 后异步清零 (asynchronous clear) 有竞争风险** (suicide counter，勿用)。

Exercises

Exercises (Past Exams & Practice)

1. **[Exam]** For an SR latch built from NOR gates, which input drives it into the invalid state?
   A. S=0, R=0 B. S=0, R=1 C. S=1, R=0 D. S=1, R=1

2. **[Exam]** Which of the following is correct?
   A. A D flip-flop can be built from combinational logic alone.
   B. In complex synchronous systems, level-triggered latches usually have better timing robustness than edge-triggered flip-flops.
   C. To realize the same function, a Moore machine generally needs more states than a Mealy machine.
   D. None of the above.

3. **[Exam]** A traffic-light controller has three states: 00 green, 01 yellow, 11 red; when input X=1 it stays on yellow. Implement it with two D flip-flops.
   (a) Write the complete state-transition table; (b) draw the state diagram; (c) use K-maps to find the next-state equations D_1, D_0; (d) draw the complete sequential circuit.

4. **[Quiz]** Design a controlled 3-bit counter with input C. Starting from 000, when $C = 1$ it repeats
   $$000 \to 100 \to 110 \to 111 \to 011 \to 000;$$
   when $C = 0$ it repeats
   $$000 \to 100 \to 110 \to 010 \to 011 \to 000.$$
   Use D flip-flops. Give the state diagram, complete state table (including unused states), simplified next-state equations, output equations, and logic circuit; state how unused states are handled.

5. **[Extra]** A sequential circuit has one D flip-flop (output S) with $D = X \oplus Y \oplus S$. Find the state table and state diagram.

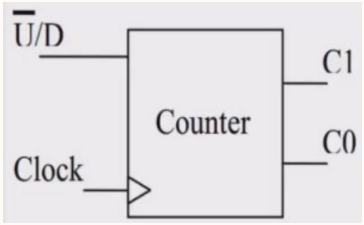
6. **[Extra]** Design a circuit with two D flip-flops A, B and input X: when X=0 hold; when X=1 cycle through $00 \to 10 \to 11 \to 01 \to 00$.

7. **[Extra]** Using D flip-flops and gates, design a counter with the cyclic sequence $0, 2, 1, 3, 4, 6, 5, 7$.

8. **[Extra]** Design a Mealy machine that detects the sequences **101** and **110**: output `10` on detecting 101, `11` on detecting 110, otherwise `00`. Draw the state diagram, state table, next-state equations, output equations, and circuit.

9. **[Extra]** Design a 2-bit up/down saturating counter (D flip-flops; the module is shown below): direction input $U/\overline{D}$ and clock, outputs C_1C_0; if $U/\overline{D}$=0 and not yet at 11 then

increment, if $U/\overline{D}$=1 and not yet at 00 then decrement. Give the state diagram, state table, simplified equations, and circuit.



10. **[Extra]** Handshake-detection FSM: synchronous inputs R, A and an asynchronous `RESET`, output E. When the handshake transitions occur in the correct order E=0; if out of order, E=1 and stays until `RESET`. Design this detector.

11. **[Extra]** Fill in the blanks: (1) The next state of a sequential circuit depends not only on the current input but also on __; (2) the setup time of a master-slave flip-flop is generally __ that of an edge-triggered flip-flop; (3) a circuit whose output depends only on the current state is a __ machine; (4) encoding 108 keyboard symbols needs at least __ binary bits; (5) for a basic SR latch built from two NOR gates, when both inputs are 0, Q __.

12. **[Extra]** Which function does a basic SR flip-flop **not** provide? A. hold B. toggle C. set D. reset.
Which one has the "ones-catching" problem? A. SR latch B. D latch C. edge-triggered D flip-flop D. SR master-slave flip-flop.

13. **[Extra]** In an 8-bit binary up-counter, how many flip-flops toggle when counting from `10100011` to the next number?

14. **[Extra]** Register micro-operations: $R1 = (10110101)_2$, $R2 = (01110110)_2$. Execute in order (a) $R1 \leftarrow sr\, R1$ (shift right) (b) $R1 \leftarrow R1 \oplus R2$; find $R1$.
74LS153 (dual 4-to-1 MUX): $I_0 \sim I_3 = 1010$, $S_1S_0 = 10$, output Y =__; for Y to be a square wave, S_1S_0 may be __.

15. **[Extra]** Modulo-counter multiple choice: (a) To design a mod-8 counter from a 4-bit synchronous binary counter, the Load signal can be asserted as: A. $\overline{Q_2Q_1Q_0}$ B. $Q_2Q_1Q_0$ C. $\sim Q_2 \sim Q_1 \sim Q_0$ D. $Q_2 + Q_1 + Q_0$. (b) For a synchronous BCD counter (Q_3 is MSB), what are the output period and positive pulse width of Q_3? A. 10 cycles / 1 B. 10 cycles / 2 C. 16 cycles / 2 D. 16 cycles / 10. (c) When designing a mod-N counter from a synchronous binary counter, Load makes the counter: A. load a nonzero initial value once B. initialize all flip-flops once C. load the initial value on the next clock edge when it reaches $N-1$ (or before start) D. none. (d) At least how many 4-bit binary counters are needed to build a counter with $N = 2010$? A. 12 B. 11 C. 3 D. 2.

16. **[Class]** Draw the logic diagram of a 4-bit register with mode-select inputs S_1, S_0 operating per the table below (build the multiplexer from "AND-gate enable + OR gate", with select control S_1S_0):

S_1	S_0	Operation
0	0	Hold (no change)
0	1	Parallel load
1	0	Shift right
1	1	Shift left

17. **[Class]** Design a register cell for an 8-bit register A that realizes the register transfers:

$$C_0:\ A \leftarrow A \cap B, \qquad C_1:\ A \leftarrow A \cup \overline{B}$$

Use AND/OR/NOT gates to find the optimal logic for the D input of the flip-flop in each cell (analyze the conflict when $C_0C_1 = 11$ and 00, and give the improved K-map and expression).

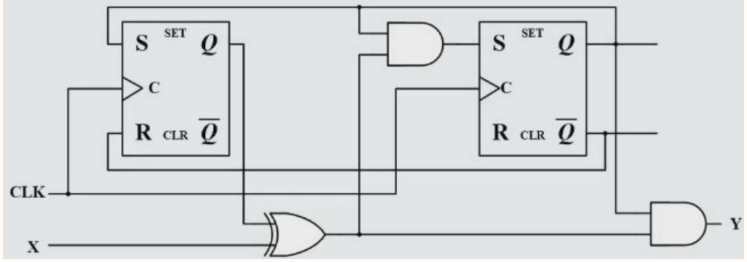
18. **[Class]** Given two register-transfer statements ($R1$ unchanged otherwise):

$$C_1:\ R1 \leftarrow R1 + R2 \text{ (add)}, \qquad C_2:\ R1 \leftarrow R1 + 1 \text{ (increment)}$$

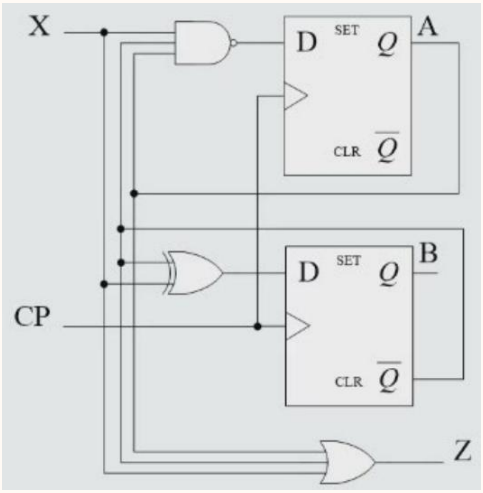
(a) Draw a logic diagram realizing both transfers using one 4-bit counter with parallel load and one 4-bit adder; (b) instead realize it using one 4-bit adder plus the necessary external gates, and compare with (a).

19. **[Class]** A sequential circuit has the device delays: inverter $t_{pd} = 0.01\,\text{ns}$; XOR gate $t_{pd} = 0.04\,\text{ns}$; flip-flop $t_{pd} = 0.08\,\text{ns}$, $t_s = 0.02\,\text{ns}$, $t_h = 0.01\,\text{ns}$. Find: (a) the longest path delay from an external input through gates only to an external output; (b) the longest path delay from an external input to a clock rising edge; (c) the longest path delay from a clock rising edge to an output; (d) the longest path delay from clock rising edge to clock rising edge; (e) the maximum operating frequency (MHz/GHz).

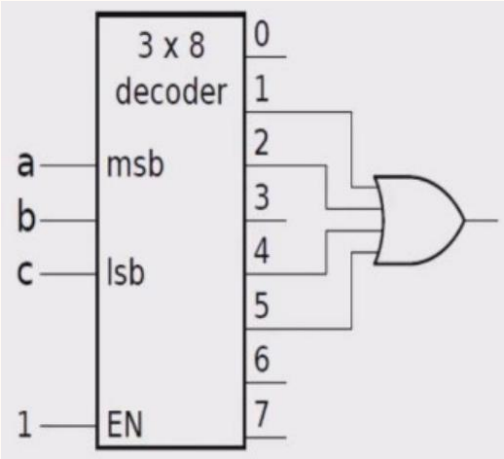
20. **[Recall]** In the sequential circuit below, XOR gate $t_{pd} = 3.0\,\text{ns}$, AND gate $t_{pd} = 1.5\,\text{ns}$, flip-flop $t_{pd} = 3.0\,\text{ns}$, $t_s = 1.0\,\text{ns}$, $t_h = 0.25\,\text{ns}$. How long before the clock rising edge must the external input be stable? A. 8.5 ns　B. 7.5 ns　C. 6.5 ns　D. 5.5 ns



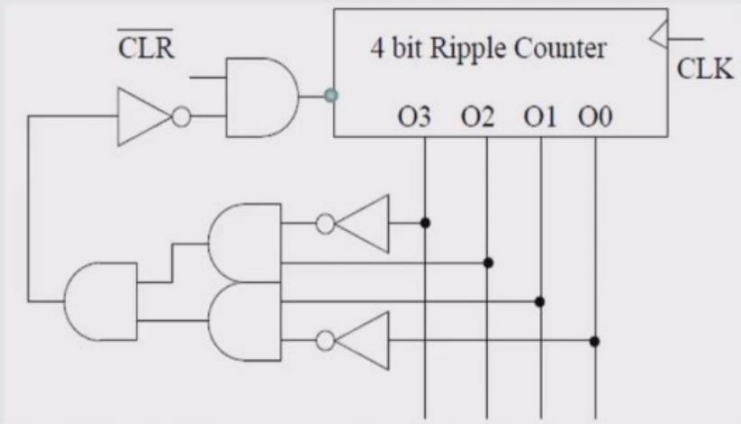
21. **[Recall]** Analyze the circuit below (two D flip-flops, input X, output Z, clock CP): write the output equation and next-state equations, build the state table, and draw the state diagram.



22. **[Recall]** What output function does the circuit below realize? (3×8 decoder, inputs a, b, c, EN=1, outputs combined through an OR gate.)



23. **[Recall]** The circuit below is a mod-N counter with a synchronous reset signal. What is N? A. 6　B. 7　C. 8　D. 9



6　指令集体系结构与寻址方式 (ISA & Addressing)

核心知识速览 (Key Concepts)

ISA 基础与设计原则 (ISA Basics & Design Principles)

- ISA (instruction set architecture) = 软硬件接口 (hardware/software interface)，定义指令、寄存器、内存模型、异常；同一 ISA 可多种实现。
- 指令 (instruction) = opcode + operands（源/目的/下一条引用）。
- 四原则 (design principles)：规则性利于简单 (simplicity favors regularity)；让常见情况更快 (make the common case fast)；更小更快 (smaller is faster)；好设计需折中 (good design demands compromise)。

操作数个数与寻址方式 (Operands & Addressing Modes)

- 操作数个数 (number of operands)：三地址 (3-address，灵活、字段多)、二地址 (2-address，短、破坏源)、一地址 (1-address，累加器 accumulator)、零地址 (0-address，栈 stack LIFO)。
- **寻址方式 (addressing modes) 与有效地址 (effective address, EA)**：立即 (immediate) $\#K$；直接 (direct) $M[K]$；间接 (indirect) $M[M[K]]$；寄存器 (register) R；寄存器间接 (register indirect) $M[R]$；变址 (indexed) $M[R_m+R_n]$；基址 + 位移 (base + displacement) $M[R_m+X]$；相对 (PC-relative) PC + offset。
- 立即数 $\neq$ 地址；间接比直接多取一层。

CISC vs RISC

- CISC：变长指令 (variable-length)、寻址丰富、微码 (microcode)、语义差距小（x86/VAX）。
- RISC：定长 (fixed-length，如 32 位)、load-store（仅 load/store 访存，ALU 只在寄存器间）、硬布线 (hardwired)、依赖编译器（MIPS/RISC-V）。
- 1980 年后新 ISA 普遍采用 load-store 风格；现代 x86 “CISC 外壳，RISC 内核”。

Exercises

Exercises (Past Exams & Practice)

1. **[Extra]** The table below lists 9 addressing modes. Write the effective-address E formula for each (Disp = displacement, B = base register, I = index register, S = scale factor, PC = program counter): (1) immediate (2) register (3) direct (4) base (5) base + displacement (6) scaled index + displacement (7) base + index + displacement (8) base + scaled index + displacement (9) PC-relative.

2. **[Extra]** A machine has instruction format: X(2 bits) | OP(6 bits) | source register(3 bits) | destination register(3 bits) | address. There are 8 general-purpose registers (16 bits each)

and a main memory of 256K words.

(1) If each memory cell is accessed directly without going through a register and OP = 6 bits, how many bits should the address field be? What should the instruction word length be?

(2) If X=11 designates a general-purpose register as a base register, give a hardware scheme that lets it access every cell of a 1M memory space.

3. **[Exam]** A 32-bit fixed-length ISA: 130 instructions, 64 general-purpose registers; each instruction has an opcode, one destination register *rd*, and one immediate *imm*, in a single word (4 bytes).

(a) Minimum bits for the opcode? (b) Minimum bits for *rd*? (c) Maximum bits for *imm*? (d) If *imm* is an unsigned byte-address offset, what is the largest representable offset? (e) If *imm* is two's complement, what are the maximum and minimum representable values?

4. **[Extra]** Address = PC plus a constant in the instruction. Which addressing mode is this? A. register B. immediate C. PC-relative D. base/displacement.

5. **[Extra]** Modern computers are built on two key principles: ① instructions are represented as numbers; ② programs can be stored in memory like data; ③ make the common case fast; ④ every instruction can be executed conditionally. The correct ones are: A. ①③ B. ②④ C. ①② D. ③④.

6. **[Extra]** True or false: (1) "Good design requires no compromise." (2) A callee-saved register is "a register saved by the program that initiates the procedure call."

7 RISC-V 汇编与机器码 (RISC-V Assembly)

核心知识速览 (Key Concepts)

寄存器与指令格式 (Registers & Instruction Formats)

- 32 个寄存器 (registers)：`x0`=0；`x1=ra` 返回地址 (return address)；`x2=sp` 栈指针 (stack pointer)；`a0-a7`(x10-17) 参数/返回值 (arguments/return)；`t*` 临时 (temporary, caller-saved, 调用不保留)；`s*` 保存 (saved, callee-saved, 调用保留)。
- 六种格式 (formats)（低 7 位 opcode）：**R** (funct7,rs2,rs1,funct3,rd)、**I** (imm[11:0],rs1,funct3,rd)、**S** (store)、**B** (branch)、**U** (lui/auipc)、**J** (jal)。
- 立即数 (immediate) 符号位恒在 inst[31]（符号扩展 sign-extended)；B/J 最低位隐含为 0（2 字节对齐 alignment)。

常用指令与控制流 (Instructions & Control Flow)

- 算术/逻辑 (arithmetic/logic)：`add/addi/sub/and/or/xor/sll/srl/sra/slt/sltu/lui/auipc`。
- 访存 (load/store, 基址 + 位移 `imm(rs1)`)：`lb/lh/lw`（符号扩展 sign-extend)、`lbu/lhu`（零扩展 zero-extend)、`sb/sh/sw`；RV64 `ld/sd/lwu`。
- 控制流 (control flow, PC 相对)：分支 (branch) `beq/bne/blt/bge/bltu/bgeu`；跳转 (jump) `jal rd,off`（$rd = PC+4$）、`jalr rd,imm(rs1)`。伪指令 (pseudo-instructions) `j`=`jal x0`, `ret`=`jalr x0,0(x1)`, `nop`=`addi x0,x0,0`。
- 寻址 (addressing)：immediate、register、base + displacement、PC-relative。

Exercises

Exercises (Past Exams & Practice)

1. **[Exam]** In a RISC-V R-type instruction, which field specifies the destination register? A. opcode B. funct3 C. rd D. rs1.
2. **[Exam]** Which format does the `jalr` instruction belong to? A. R B. I C. S D. J (or U).
3. **[Exam]** Memory contents are as follows, with initial `R2 = 0x0004`:

Address	Data
0x1000	0x1008
0x1004	0x100C
0x1008	0x1004
0x100C	0x100D

After executing `lw R1, 0x1004(R2)`, what is the value of `R1`? A. 0x1004 B. 0x1008 C.

0x100C D. 0x100D.

4. **[Extra]** Give the instruction type and the assembly instruction for the following binary: `0000 0000 0001 0000 1000 0000 1011 0011`

5. **[Extra]** Translate the function `f` into RISC-V assembly (`g` is declared as `int g(int a,int b)`):

```
int f(int a, int b, int c, int d){
  return g(g(a,b), c+d);
}
```

6. **[Extra]** Expand the following pseudo-instructions into the **shortest** RISC-V instruction sequence:

Pseudo-instruction	Function
`li rd, BIG`	`rd = BIG` (32-bit two's complement constant)
`sne rd, rs1, rs2`	`rd = (rs1 != rs2) ? 1 : 0`

7. **[Extra]** Give the hexadecimal machine code of the following instructions (starting memory address `0x0040 0000`):

```
    ld  x12, 8(x10)
loop:  bne  x12, x13, label
    slli  x13, x12, 4
    jal  x0, loop
label:  jalr  x0, 0(x1)
```

8. **[Extra]** Decode `0x00578833` into a RISC-V assembly instruction (write out each field).

9. **[Quiz]** The same 32-bit pattern `0xFF53AE93` may be interpreted in several ways. Decode it as (a) a 32-bit two's-complement integer, (b) an IEEE 754 single-precision value, and (c) an RV32I instruction. Show all field boundaries and intermediate values.

10. **[Quiz]** The instruction at address `0x00200000` is `Loop: jal x0, L1`; the instruction at `0x00200008` is `beq x3, x4, Loop`; label `L1` is at `0x00200070`. Encode both instructions as 32-bit hexadecimal machine code.

11. **[Quiz]** Convert each pseudoinstruction to the shortest RV32I sequence: (a) `bnez rs, Label`; (b) `j Label`; (c) `mv rd, rs`; (d) `not rd, rs`. Then write the shortest branch sequence to label `IndexOutOfBounds` if $x20 \geq x11$ in signed comparison or if $x20$ is negative.

8 处理器性能与单周期 CPU (Performance & Single-Cycle)

核心知识速览 (Key Concepts)

性能公式

$$\text{CPU Time} = IC \times CPI \times T_c = \frac{IC \times CPI}{\text{Clock Rate}}$$

- 加权 (weighted) $CPI = \sum CPI_i \times \dfrac{IC_i}{IC}$ (IC = instruction count, CPI = cycles per instruction)。
- "X 比 Y 快 n 倍" $\Leftrightarrow \dfrac{\text{Time}_Y}{\text{Time}_X} = n$。
- **Amdahl 定律 (Amdahl's law)**: $\text{Speedup} = \dfrac{1}{(1-F_e) + F_e/S_e}$, 上界 $\dfrac{1}{1-F_e}$; 优先优化占比最大部分。
- 三堵墙 (three walls): 功耗墙 (power wall, $P \propto CV^2 f$)、存储墙 (memory wall)、ILP 墙 $\Rightarrow$ 多核 (multicore)。

单周期数据通路与控制 (Single-Cycle Datapath & Control)

- 五阶段 (five stages): IF → ID → EX → MEM(仅 lw/sw) → WB; 指令存储器 (instruction memory) 与数据存储器 (data memory) **分开 (separate)**。
- 主控制信号 (control signals): ALUSrc、MemtoReg、RegWrite、MemWrite、MemRead、Branch、Jump + ALUOp。
- 两级译码: 主译码器 (main decoder, opcode)→ 控制信号 +ALUOp; ALU 译码器 (ALU decoder, ALUOp+funct)→ ALU 控制。
- 时钟周期 (clock cycle) = 关键路径 (critical path) (通常 Load: 取指 200+ 读寄存器 100+ALU200+ 读内存 200+ 写寄存器 100 = 800ps)。

单周期控制信号真值表 Control Signal Truth Table (记忆用)

Instr	ALUSrc	MemtoReg	RegWR	MemWR	Branch	Jump	ALUOp
R-type	0	00	1	0	0	0	10
Load (ld)	1	01	1	0	0	0	00
Store (sd)	1	X	0	1	0	0	00
Branch(beq)	0	X	0	0	1	0	01
Jump (jal)	X	10	1	0	0	1	XX
I-ALU(addi)	1	00	1	0	0	0	11

Exercises

Exercises (Past Exams & Practice)

1. **[Exam]** A single-cycle CPU supports four instruction classes A, B, C, D with frequencies 10%/20%/30%/40% and CPIs 1, 4, 3, ? respectively. After optimization, D's CPI is halved and the overall CPI becomes 3. Find D's original CPI. A. 4 B. 5 C. 6 D. 8.

2. **[Exam]** Which component is **required** in a multi-cycle CPU but **not strictly required** in a single-cycle CPU?
   A. instruction register IR B. program counter PC C. instruction memory D. multiplexer MUX.

3. **[Exam]** Single-cycle CPU execution times by instruction class: R-type 750ps, Load 1000ps, Branch 875ps, I-type 625ps, Jump 600ps, each class occurring with equal frequency.
   (a) Find the clock cycle time and clock frequency; (b) find the average CPI; (c) find the CPU execution time for 10^8 instructions; (d) if a redesign reduces the time in (c) to 80% of the original but increases the total cycle count by 20%, find the new clock frequency.

4. **[Extra]** P1, P2, P3 run the same instruction set: P1 2GHz/CPI 1.5, P2 1.5GHz/CPI 1.0, P3 3GHz/CPI 2.5.
   (1) Which has the highest performance? (2) If each runs a program for 10 seconds, find the number of clock cycles and instructions for each.

5. **[Extra]** CISC vs RISC performance: a task needs P CISC instructions or $2P$ RISC instructions; one CISC instruction takes $8T$ ns, one RISC instruction takes $2T$ ns. Which performs better?

6. **[Extra]** A function is sped up 20× but accounts for only 40% of total run time; what is the overall speedup?
   If adding an FP unit speeds up FP operations 20× and the overall speedup is 5×, find the FP fraction F_e.

7. **[Extra]** Translate the following C functions into RISC-V assembly callable from a standard C program:

   ```
   void main(int *z, int x, int y){ *z = sub(x, y); }
   int sub(int n, int m){ return m - n; }
   ```

   Then: (1) In a single-cycle design, can the instruction memory IM and data memory DM be merged into one MEM? If yes, draw the datapath for `lw/sw`; if not, explain why. (2) Given IM/DM read 2ns and write 1.5ns, control unit 4.6ns, ALU 1.7ns, register file 1.3ns, find the optimal clock cycle time.

8. **[Extra]** Add a register–memory instruction `Addm rd, rs, rt` (`rd = rs + Mem[rt]`) to

the single-cycle datapath.

(a) Give the binary instruction format; (b) describe the datapath changes needed and draw the datapath for this instruction; (c) list all control signals and their values for this instruction.

9. **[Extra]** Examine the difficulty of adding `ss rs1, rs2, imm` (Store Sum: `Mem[Reg[rs1]] = Reg[rs2] + imm`) to RISC-V: (a) What new functional units are needed? (b) Which existing units must change? (c) What new datapaths are needed? (d) What new signals does the control unit need? (e) Sketch the implementation on the single-cycle datapath.

10. **[Exam]** A processor with separate instruction/data memories defines three instructions (word = 4 bytes):

Instruction	Semantics
`lw ri, imm`	`Reg[ri]=Mem[imm]`, `PC=PC+4`
`sw ri, imm`	`Mem[imm]=Reg[ri]`, `PC=PC+4`
`add ri, imm`	`Reg[ri]=Reg[ri]+imm`, `PC=PC+4`

(a) Complete the interface ports of the register file and data memory; (b) draw the complete datapath supporting all three instructions (you may add MUXes, sign extension, etc.); (c) design the control signals for the three instructions and give each signal's value in a table.

11. **[Quiz]** A single-cycle processor runs at 4 GHz and executes 100,000 instructions in 0.03 ms. Find its CPI. A new processor executes instruction classes A, B, C with counts $1500, 2000, 500$ and CPIs equal to the previous answer, 1.4, and 2.1, respectively; find its average CPI. Finally, for a multi-cycle RISC-V CPU whose R-type/Load/Store/Branch/Jump instructions require 4/5/4/3/3 cycles and occur with percentages 50%/10%/10%/15%/15%, find the average CPI and total cycle count for 10^6 instructions.

9　多周期处理器 (Multi-Cycle Processor)

核心知识速览 (Key Concepts)

多周期要点 (Multi-Cycle Essentials)

- 一个时钟周期执行指令的一**个阶段 (one stage)**；时钟周期由**最复杂的单步**决定（而非最长指令）。
- 数据通路变化：**合并存储器 (unified memory)**（指令/数据共用）、**合并 ALU**（去掉额外加法器）、**新增 5 个临时寄存器 (temporary registers)**：IR、MDR、A、B、ALUout。
- 阶段数 (number of stages)：beq/j = 3；R 型/Store = 4；Load = 5。
- 控制信号**随阶段变化** ⇒ 控制器是**时序逻辑 (sequential logic)**（需记当前阶段）。
- 控制器实现：**硬连线 (hardwired)**（FSM+PLA，适合 RISC）vs **微程序 (microprogrammed)**（控制存储器 + 微指令，适合 CISC）。
- 性能：加权 CPI（例 ≈ 4.12），T_c 由存储器读主导（例 325ps）；简单实现下**多周期不一定快于单周期**（时序开销多次支付）；其意义是为流水线 (pipelining) 奠基。

Exercises

Exercises (Past Exams & Practice)

1. **[Extra]** List the 5 new temporary registers added in a multi-cycle CPU and the role of each, and state the number of stages and the stage sequence required by each instruction class (beq, R-type, sw, lw).
2. **[Extra]** Given the cycle counts per instruction class in a multi-cycle design: branch/jump 3, R-type/Store 4, Load 5; and the instruction mix Load 25%, Store 10%, Branch 11%, Jump 2%, R-type 52%. Find the weighted average CPI.
3. **[Extra]** Let the multi-cycle design have $T_c = 325$ps, CPI $= 4.12$, and the single-cycle design have $T_c = 925$ps, CPI $= 1$. Find the time to execute 10^{11} instructions for each, and explain why the single-cycle design is actually faster in this example.
4. **[Extra]** Compare a hardwired control unit with a microprogrammed control unit: what are the implementation method, applicable scenario (RISC/CISC), and pros and cons of each?

"Follow the main thread when reviewing: how data is encoded → how logic computes → how state is stored → how instructions are defined → how the processor executes."